

REST Assured API Automation Test Report

Prepared by: Justin Philip

Date: 25 June 2026

GitHub Repository:

- API Automation Project:
-

1. Objective

The objective of this automation is:

Create a user, generate an authentication token and retrieve the list of books using token.

2. Application / API Under Test

- API Type: RESTful APIs
- Base URL: <https://demoqa.com>
- Environment: Local/ CI/CD

APIs Covered:

- Creating User: <https://demoqa.com/Account/v1/User>
 - Generating Token: <https://demoqa.com/Account/v1/GenerateToken>
 - Getting all Books: <https://demoqa.com/BookStore/v1/Books>
-

3. Environment & Setup

Technology Stack:

- Language: Java
- Framework: Rest Assured
- Assertion Library: JUnit 5
- Build Tool: Maven
- Test Runner: Maven Surefire Plugin
- Environmental Variables: stored in .env file(local)/ secrets(CI/CD)

4. Framework Design

4.1 Architecture Overview

The framework is designed using a structured approach separating test logic and API interaction layers.

4.2 Project Structure

- src/test/java/tests → Test classes
 - client/ → Clients for the different api response
 - contracts/ → contracts for the json schema validation
 - data/ → stores testData and requestBodies for client Requests
 - model/ → stores various response models of apis
 - specFactory/ → Standard Specification for Requests and Responses
 - support/ → set up to call env variables
 - resources/ → Test data and config
 - pom.xml → Dependency management
-

5. Test Approach / Strategy

- API testing is conducted using functional validation
- Focus on:
 - Status code validation
 - Response body verification
 - Schema validation

Test Category:

- Smoke → Basic API availability
-

7. Scenarios Covered

- POST for creating User

- POST for generating Token after Authentication
 - GET method for retrieving all books
-

8. Test Scenario

Scenario

- **Scenario Name:** Creating User, generating Token and retrieving Books

Steps:

1. Send api POST request to “/Account/v1/User”
2. Validate json schema and response body
3. Send POST request to generate token to “/Account/v1/GenerateToken”.
4. Validate json schema and response body
5. Send GET request to “/BookStore/v1/Books”
6. Validate json schema and response body

Expected Result:

- API should return status code 201 at first instance of POST
- API should return status code 200 at second instance of POST
- API should return status code 200 for GET alongwith response

Actual Result:

- API returned status code 201 at first instance of POST
- API returned status code 200 at second instance of POST
- API returned status code 200 for GET

Status: Pass

Code:

```
public void createUserWithPOST(){
    //Creating User
    Response response = (Response) new UserClient().user().then()
        .spec(new SpecBuilder().respSpec())
        .extract().response();

    //Schema Assertion
    assertCreatedJsonContract(response, schemaPath: "schemas/user.schema.json");

    SERIALIZATION
    UserRequest userDetails = response.then()
        .extract().as(UserRequest.class);

    //Response Assertion
    assertNotNull(userDetails.userID());
    assertNotNull(userDetails.username());
    assertTrue(userDetails.books().isEmpty());
}
```

```
//Schema Assertion
assertOkJsonContract(resp, schemaPath: "schemas/token.schema.json");

SERIALIZATION
Token_AuthZ userToken = resp.then()
    .extract().as(Token_AuthZ.class);

//Response Assertion
assertFalse(userToken.token().isBlank());
assertTrue(userToken.status().equals("Success"));
assertTrue(userToken.result().contains("User authorized successfully"));

//Getting all Books
Response resp_books = (Response) new BookListClient()
    .bookList(userToken.token()) Response
    .then().spec(new SpecBuilder().respSpec()) ValidatableResponse
    .statusCode(200).extract().response();

//Schema Assertion
assertOkJsonContract(resp, schemaPath: "schemas/bookcheck.schema.json");
```

```
SERIALIZATION
BookListModel books = resp_books.then()
    .extract().as(BookListModel.class);

assertFalse(books.books().get(0).isbn().isEmpty());
assertFalse(books.books().get(0).title().isBlank());
assertFalse(books.books().get(0).subTitle().isEmpty());
assertFalse(books.books().get(0).author().isBlank());
assertFalse(books.books().get(0).publish_date().isEmpty());
assertFalse(books.books().get(0).publisher().isBlank());
assertTrue(condition: books.books().get(0).pages()>0);
assertFalse(books.books().get(0).description().isBlank());
assertFalse(books.books().get(0).website().isEmpty());
```

Conclusion:

The API responses of the three given endpoints are validated and found correct.